Programming Codes

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <iomanip>
#include <algorithm>
#include <ctime>
#include <limits>
using namespace std;
// ============================================================
// UTILITY FUNCTIONS
// ============================================================
string getCurrentDate() {
time_t now = time(0);
tm* ltm = localtime(&now);
ostringstream oss;
oss << (1900 + ltm->tm_year) << "-"
<< setw(2) << setfill('0') << (1 + ltm->tm_mon) << "-"
<< setw(2) << setfill('0') << ltm->tm_mday;
return oss.str();
}
void printLine(char ch = '-', int width = 75) {
cout << string(width, ch) << endl;
```

```
}
void clearInputBuffer() {
cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
// Safe input
int getIntInput() {
int x;
while (!(cin >> x)) {
cout << "Invalid input. Enter number: ";
cin.clear();
clearInputBuffer();
}
clearInputBuffer();
return x;
}
double getDoubleInput() {
double x;
while (!(cin >> x)) {
cout << "Invalid input. Enter number: ";
cin.clear();
clearInputBuffer();
}
clearInputBuffer();
return x;
}
```

```
// ============================================================

// BASE CLASS

// ============================================================


class Medicine {

protected:

int id;

string name, category, manufacturer, expiryDate, batchNo;

double price;

int quantity;


public:

Medicine() : id(0), price(0), quantity(0) {}


Medicine(int id, string n, string c, string m,

double p, int q, string e, string b)

: id(id), name(n), category(c), manufacturer(m),

price(p), quantity(q), expiryDate(e), batchNo(b) {}

virtual ~Medicine() {

int getId() const { return id; }

string getName() const { return name; }

string getCategory() const { return category; }

double getPrice() const { return price; }

int getQuantity() const { return quantity; }

string getExpiryDate() const { return expiryDate; }

string getBatchNo() const { return batchNo; }
```

```
    void setQuantity(int q) { quantity = q; }
    virtual string getType() const { return "General"; }
    virtual void displayRow() const {
        cout << left
             << setw(5) << id
             << setw(20) << name
             << setw(15) << category
             << setw(10) << getType()
             << setw(8) << quantity
             << setw(10) << price
             << setw(12) << expiryDate
             << endl;
    }
    virtual string toCSV() const {
        return std::to_string(id) + "," + name + "," + category + "," +
               manufacturer + "," + std::to_string(price) + "," +
               std::to_string(quantity) + "," + expiryDate + "," +
               batchNo + "," + getType();
    }
};

// ============================================================
// DERIVED CLASSES
// ============================================================

class PrescriptionMedicine : public Medicine {
    string doctor, schedule;
```

```
public:
PrescriptionMedicine(int id, string n, string c, string m,
double p, int q, string e, string b,
string d, string s)
: Medicine(id, n, c, m, p, q, e, b), doctor(d), schedule(s) {}
string getType() const override { return "Rx"; }
string toCSV() const override {
return Medicine::toCSV() + "," + doctor + "," + schedule;
}
};
class OTCMedicine : public Medicine {
string usage;
public:
OTCMedicine(int id, string n, string c, string m,
double p, int q, string e, string b, string u)
: Medicine(id, n, c, m, p, q, e, b), usage(u) {}
string getType() const override { return "OTC"; }
string toCSV() const override {
return Medicine::toCSV() + "," + usage;
}
};
// ============================================================
// TRANSACTION STRUCT
// ============================================================
struct Transaction {
string date, type, name;
```

```
int id, qty;
double price, total;
string toCSV() const {
return date + "," + type + "," + to_string(id) + "," +
name + "," + to_string(qty) + "," +
to_string(price) + "," + to_string(total);
}
};
// ============================================================
// MAIN CLASS
// ============================================================
class Pharmacy {
vector<Medicine*> inventory;
vector<Transaction> transactions;
int nextId = 1;
Medicine* find(int id) {
for (auto m : inventory)
if (m->getId() == id) return m;
return nullptr;
}
public:
~Pharmacy() {
for (auto m : inventory) delete m;
}
void addMedicine() {
cout << "\n1.Rx 2.OTC 3.General: ";
```

```
int t = getIntInput();
string name, cat, mfr, exp, batch;
cout << "Name: "; getline(cin, name);
cout << "Category: "; getline(cin, cat);
cout << "Manufacturer: "; getline(cin, mfr);
cout << "Price: "; double price = getDoubleInput();
cout << "Qty: "; int qty = getIntInput();
cout << "Expiry: "; getline(cin, exp);
cout << "Batch: "; getline(cin, batch);
Medicine* m;
if (t == 1) {
string d, s;
cout << "Doctor: "; getline(cin, d);
cout << "Schedule: "; getline(cin, s);
m = new PrescriptionMedicine(nextId, name, cat, mfr, price, qty, exp, batch, d, s);
}
else if (t == 2) {
string u;
cout << "Usage: "; getline(cin, u);
m = new OTCMedicine(nextId, name, cat, mfr, price, qty, exp, batch, u);
}
else {
m = new Medicine(nextId, name, cat, mfr, price, qty, exp, batch);
}
inventory.push_back(m);
cout << "Added ID: " << nextId++ << endl;
```

```
}
void view() {
printLine('=');
for (auto m : inventory)
m->displayRow();
printLine('=');
}
void sale() {
cout << "ID: ";
int id = getIntInput();
Medicine* m = find(id);
if (!m) return;
cout << "Qty: ";
int q = getIntInput();
if (q > m->getQuantity()) {
cout << "Not enough stock\n";
return;
}
m->setQuantity(m->getQuantity() - q);
double total = q * m->getPrice();
cout << "TOTAL: " << total << endl;
}
};
// ============================================================
// MAIN
// ============================================================
```

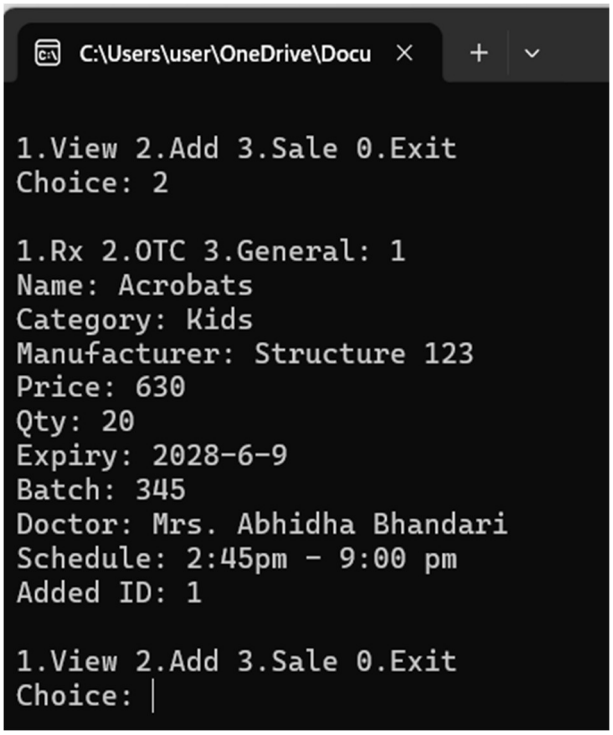
```
int main() {
Pharmacy p;
int ch;
do {
cout << "\n1.View 2.Add 3.Sale 0.Exit\nChoice: ";
ch = getIntInput();
switch (ch) {
case 1: p.view(); break;
case 2: p.addMedicine(); break;
case 3: p.sale(); break;
}
} while (ch != 0);
return 0;
```

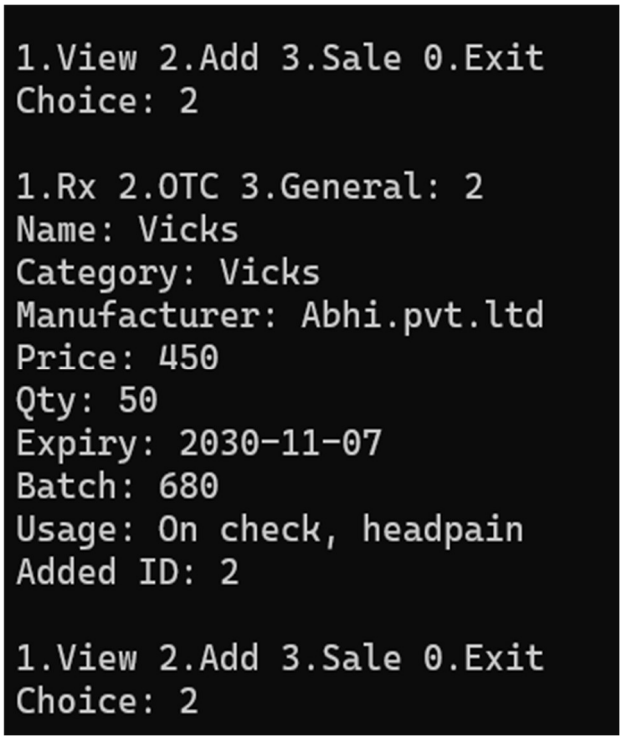
```
}
```

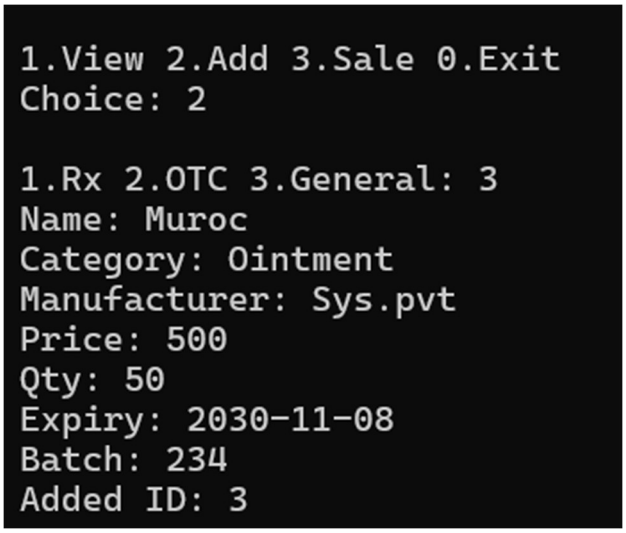
Screenshots of what I have done

1) First adding the medicine: In the terms of Rx, OTC, and general

Using three different numbers and adding in name, category, manufacture, price, quantity, expiry date, batch, usage, and added id. In this Rx means – prescription drugs with doctor prescription, OTC means – Over-the-Counter meds without doctor prescription, and General means commonly stocked items in the pharmacy.







2) Viewing

In this it shows what inputs have we written in the option to add.

```
1.View 2.Add 3.Sale 0.Exit
Choice: 1
================================================================================
1    Acrobats            Kids           Rx        20      630       2028-6-9
2    Vicks               Vicks          OTC       50      450       2030-11-07
3    Muroc               Ointment       General   50      500       2030-11-08
================================================================================
```

3) Sale

In this it to put what we have sold

```
1.View 2.Add 3.Sale 0.Exit
Choice: 3
ID: 1
Qty: 2
TOTAL: 1260

1.View 2.Add 3.Sale 0.Exit
Choice: 3
ID: 2
Qty: 3
TOTAL: 1350

1.View 2.Add 3.Sale 0.Exit
Choice: 3
ID: 3
Qty: 3
TOTAL: 1500
```

```
1.View 2.Add 3.Sale 0.Exit
Choice: 1
================================================================================
1    Acrobats            Kids           Rx        18      630       2028-6-9
2    Vicks               Vicks          OTC       47      450       2030-11-07
3    Muroc               Ointment       General   47      500       2030-11-08
================================================================================
```

4) Exit option

By entering 0 we can exit the program.

```
1.View 2.Add 3.Sale 0.Exit
Choice: 0

Process returned 0 (0x0)   execution time : 5123.595 s
Press any key to continue.
```